

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

```
In [3]: conn = sqlite3.connect('customer_churn.db')

sql_query = """SELECT name FROM sqlite_master WHERE type='table'"""

tables = pd.read_sql_query(sql_query, conn)

for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

Created dataframe: df_db_customer

Created dataframe: df_db_subscription

Created dataframe: df_db_support

```
In [4]: # Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

Table Name: db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

Data Cleaning

```
In [5]: df_db_customer.head()
```

Out[5]:

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [6]: df_db_customer.tail()

Out[6]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [7]: df_db_customer.info()

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   customerid  21 non-null    str
 1   name        21 non-null    str
 2   country     18 non-null    str
 3   state       21 non-null    str
 4   gender      21 non-null    str
 5   dob         21 non-null    str
 6   interests   4 non-null     str
 7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB

```

```
In [8]: df_db_customer.isnull().sum()
```

```

Out[8]: customerid    0
        name          0
        country       3
        state         0
        gender        0
        dob           0
        interests     17
        pincode       21
        dtype: int64

```

```

In [9]: # a. rename col - name to customer_name
        # b. drop columns - interest and pincode
        # c. change data type - dob
        # d. data standardization - gender
        # e. fix missing values (using existing data) - country

```

```

In [10]: # a. rename col - name to customer_name

df_db_customer.rename(columns={'name': 'customer_name'}, inplace=True)

```

```

In [11]: # b. drop columns - interest and pincode

df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)

```

```

In [12]: # c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])

```

```
In [13]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB
```

In [14]: *# d. data standardization - gender*

```
df_db_customer['gender'].unique()
```

Out[14]: <StringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str

In [15]: `df_db_customer['gender'] = df_db_customer['gender'].replace({'Men': 'Male', 'Wom`

In [16]: `df_db_customer.tail()`

Out[16]:

	customerid	customer_name	country	state	gender	dob
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06

In [17]: *# e. fix missing values (using existing data) - country*

```
# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]
```

Out[17]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [18]: *# country and state - unique value pair*

```
# Creating state → country map from non-null rows
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('sta

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
```

```
In [19]: df_db_customer[df_db_customer['country'].isna()] # no null value in country column
```

```
Out[19]:
```

	customerid	customer_name	country	state	gender	dob
--	------------	---------------	---------	-------	--------	-----

```
In [20]: df_db_customer[['country', 'state']]
```

```
Out[20]:
```

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	India	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	Nepal	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	India	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

```
In [21]: df_db_subscription.head()
```

Out[21]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
--	------------	-------------------------	-------------------	--------------	-----------	------

0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	



In [22]: `df_db_subscription.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                            21 non-null     str
1   subscription_start_date                21 non-null     str
2   subscription_type                      21 non-null     str
3   renewal_date                          21 non-null     str
4   plan_type                             21 non-null     str
5   contract_type                         21 non-null     str
6   cancellation_date                      6 non-null      str
7   cancellation_reason                    6 non-null      str
8   monthly_charges                       21 non-null     float64
9   cltv                                  21 non-null     int64
10  churn_score                            21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

```
In [23]: date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)

df_db_subscription.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date              21 non-null     datetime64[us]
2   subscription_type                     21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                   6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                 21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB

```

In [24]: `df_db_support.head()`

Out[24]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [25]: `df_db_support.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column            Non-Null Count  Dtype
---  -
0   customerid        9 non-null     str
1   complaint_date     9 non-null     str
2   escalations        9 non-null     str
3   csat_score         9 non-null     int64
4   col_1              0 non-null     object
5   comment            4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes

```

In [26]: `df_db_support.drop(columns=['col_1', 'comment'], inplace=True)`

In [27]: `df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])`
`df_db_support.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            9 non-null     str
1   complaint_date        9 non-null     datetime64[us]
2   escalations           9 non-null     str
3   csat_score            9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

Feature Engineering & Data Analysis

Create a new column

```
In [28]: # create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].isnull(), 0, 1)
df_db_subscription.head()
```

```
Out[28]:
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

```
In [ ]: # first fix support table duplicates then merge

df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid', how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left') )
```

```
In [30]: df_db_subscription.shape
```

```
Out[30]: (21, 12)
```

```
In [31]: df.shape
```

```
Out[31]: (23, 20)
```

```
In [32]: df_db_support['customerid'].nunique() #7

df_db_support['customerid'].size #9 Problem is here, there are 2 duplicate rows
```


Out[32]: 9

In [33]: df_db_support

Out[33]:

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

In [35]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].count()
df_db_support

Out[35]:

	customerid	complaint_date	escalations	csat_score	complaint_count
0	0003-MKNFE	2024-08-28	N	60	2
1	0003-MKNFE	2024-08-28	Y	10	2
2	0013-EXCHZ	2024-01-20	Y	20	1
3	0013-MHZWF	2025-03-18	N	90	1
4	0013-SMEOE	2024-11-01	N	30	1
5	0017-IUDMW	2024-04-10	Y	25	1
6	0019-EFAEP	2024-09-27	Y	30	1
7	0022-TCJCI	2024-09-13	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2

In [36]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid')

In [37]: df_db_support['customerid'].size

Out[37]: 7

In [38]: # merge df
df = (df_db_subscription
 .merge(df_db_customer, on = 'customerid', how= 'left')
 .merge(df_db_support, on = 'customerid', how= 'left'))

In [39]: df.shape

Out[39]: (21, 21)

```
In [40]: # export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

Data Analysis

```
In [42]: df.columns
```

```
Out[42]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
              dtype='str')
```

```
In [ ]: # 1. Churn Rate
```

```
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [44]: # 2. Retention Rate
```

```
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [48]: df.head(3)
```

```
Out[48]:
```

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
--	------------	-------------------------	-------------------	--------------	-----------	------

0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	

3 rows × 21 columns



```
In [49]: # 3. Churn by Plan Type
```

```
churn_by_plan = df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).r
print(churn_by_plan)
```

	plan_type	churn_by_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [56]: # 4 a. Churn by State
```

```
churn_by_state = df.groupby('state')['churn_flag'].mean().mul(100).round(2).reset_index()
print(churn_by_state, "Total Sum is: ", sum(churn_by_state['churn_by_pct']), "Total Count is: ")
```

	state	churn_by_pct
0	Delhi	25.00
1	Karnataka	100.00
2	Kathmandu	0.00
3	Maharashtra	0.00
4	Meghalaya	66.67
5	Nagaland	0.00
6	Rajasthan	0.00
7	Telangana	50.00
8	Uttar Pradesh	0.00

Total Sum is: 241.67000000000002 Total Count is: 9

In []: *# 4 b. Churn by Subscription type*

```
churn_by_subscription_type = df.groupby('subscription_type')['churn_flag'].mean().mul(100).round(2).reset_index()
print(churn_by_subscription_type, "Total Sum is: ", sum(churn_by_subscription_type['churn_by_pct']), "Total Count is: ")
```

	subscription_type	churn_by_pct
0	Organic	0.00
1	Paid	16.67
2	Refferal	83.33

Total Sum is: 100.0 Total Count is: 3

In [58]: *# 5. ARPU - Avg Revenue per user*

```
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

ARPU = 18.85

In [59]: df.columns

Out[59]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count'], dtype='str')

In []: *# 5. Calculating customer_age*

```
# today = pd.to_datetime('today')
# today

# np.where(
#     df['dob'].notna(), (today - df['dob']).dt.days // 365, np.nan
# )
```

Out[]: array([44., 30., 48., 24., 36., 37., 49., 27., 41., 32., 29., 45., 21., 34., 46., 40., 31., 25., 42., 35., 48.])

In [72]: *# 6. Avg Customer Tenure*

```
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()
today

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),
```

```

(df['cancellation_date'] - df['subscription_start_date']).dt.days,

(today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print('Average Tenure = ', round(avg_tenure,0), 'days')

```

Average Tenure = 1495.0 days

In [73]: # 7. Revenue at risk - revenue lost from churned users

```

revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)

```

Revenue at Risk (Rs 'K') = 73.94

In [76]: # 8. Escalation Rate

```

escalation_rate = (df['escalations']=="Y").mean()*100
print("Escalation Rate = ", round(escalation_rate,2), "%")

```

Escalation Rate = 19.05 %

In [78]: # 9. Avg Complaint Per User

```

avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))

```

Avg Compliants Per User = 0.43

In [79]: # 10. Correlation Escalation vs Churn

```

df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string to int
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation,2))

```

Correlation between escalation vs churn is = 0.77

In [80]: # 11. Create a column using existing col - Churn risk

```

conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50 & (df['churn_score'] < 70)),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unkown')

```

In [81]: df.head()

Out[81]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

5 rows × 23 columns

◀ ◯ ▶

Visualization using Matplotlib

In [82]: `df_visual = df.copy()`

In [83]: `df_visual.shape`

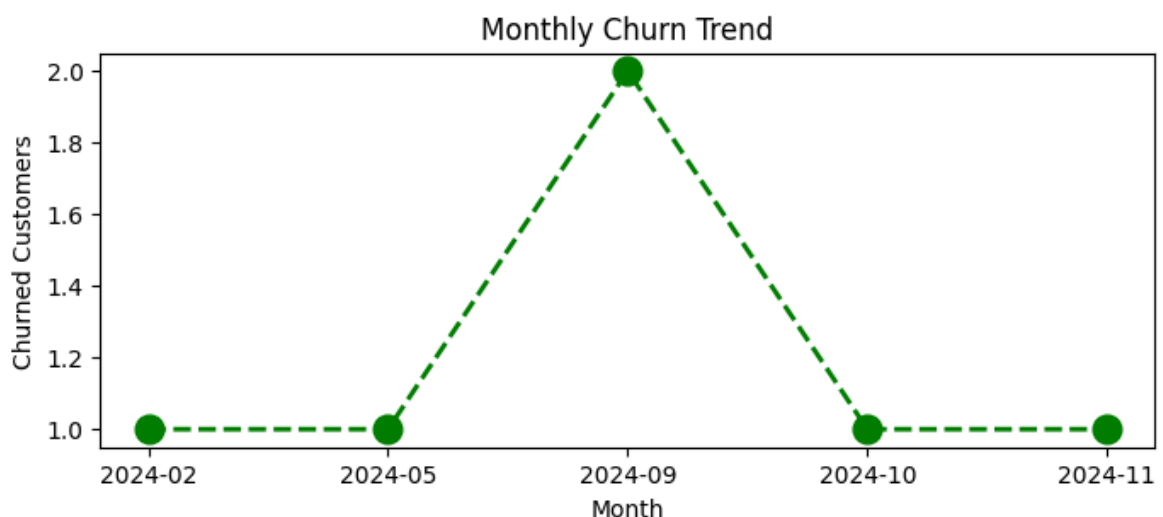
Out[83]: (21, 23)

In [84]: `# Monthly Churn Trend (Time Series KPI)`

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker='o')

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

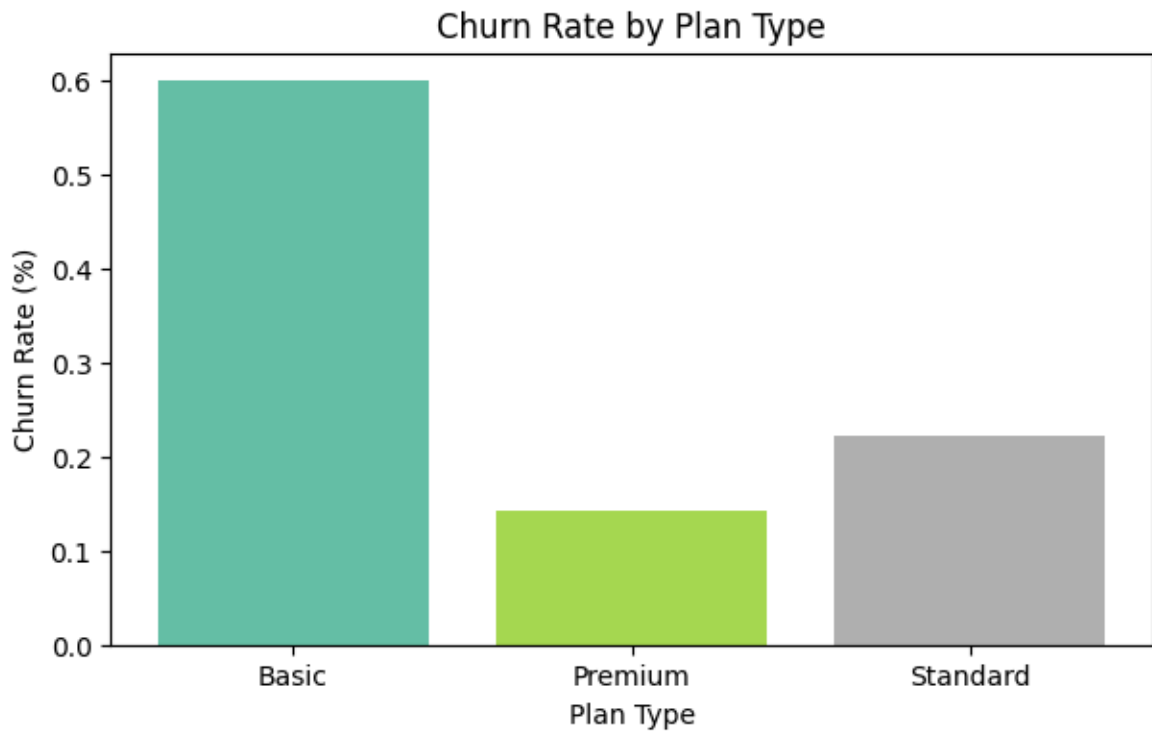


```
In [85]: # Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```

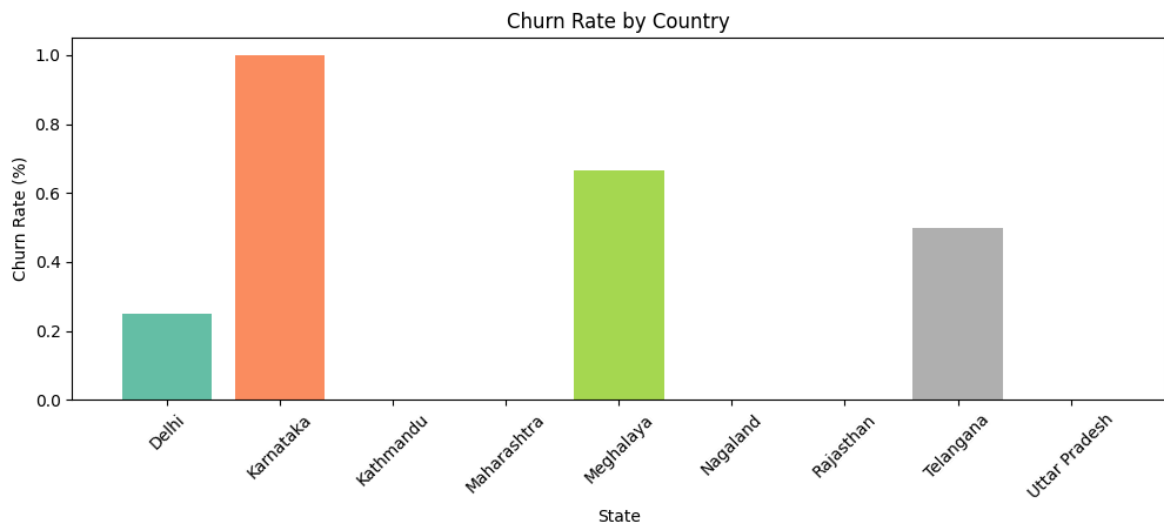


```
In [86]: # Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



Visulaization using Seaborn

```
In [87]: # Heatmap (Correlation Matrix)
```

```
In [ ]: # encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

```
Out[ ]: Index(['customerid', 'subscription_start_date', 'subscription_type',
              'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
              'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
              'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
              'complaint_date', 'escalations', 'csat_score', 'complaint_count',
              'tenure_days', 'churn_risk', 'cancellation_month'],
              dtype='str')
```

```
In [ ]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]
```

```
Out[ ]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

```
In [99]: # remove warnings
import warnings
warnings.filterwarnings("ignore")
```

```
In [97]: # incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]

categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

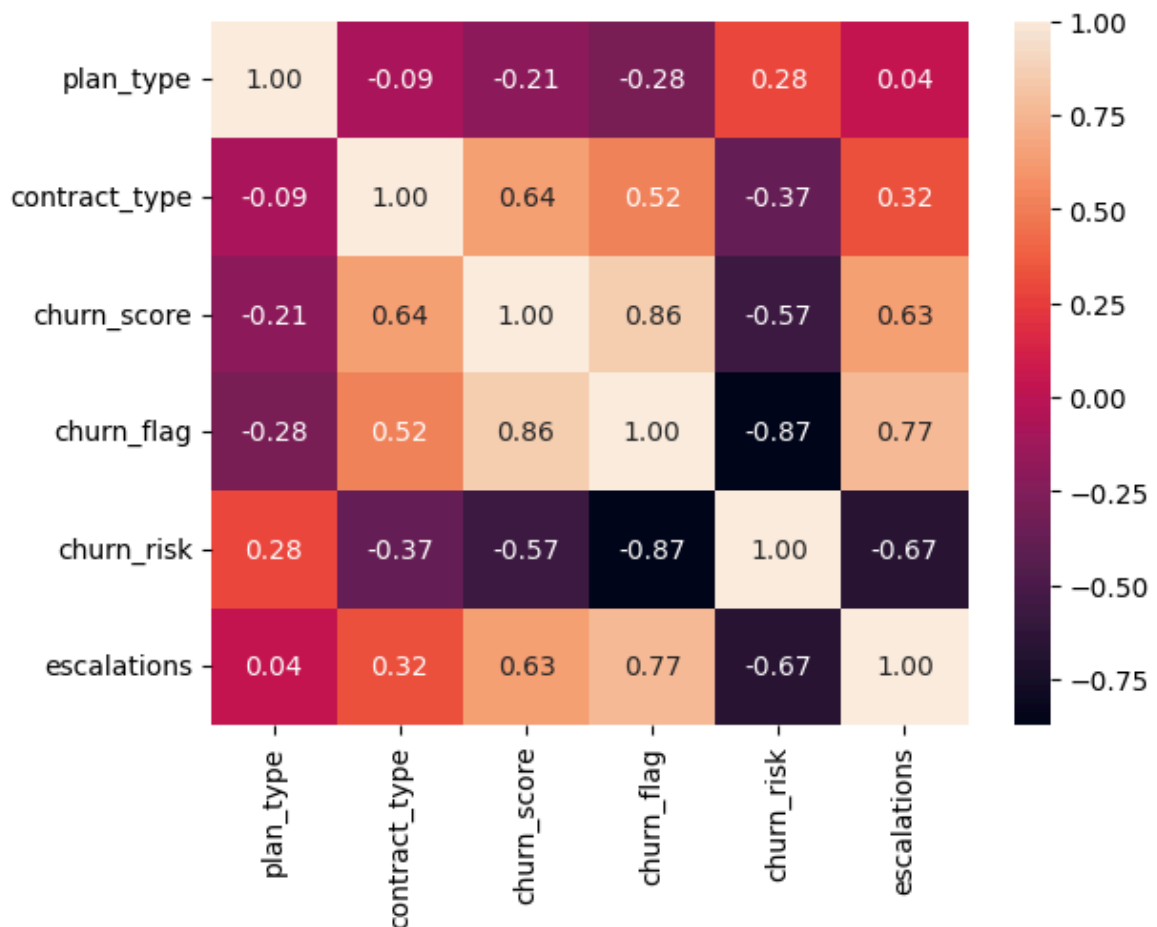
```
In [98]: df_encoded.head()
```

```
Out[98]:
```

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	2	0	12	0	1	0
1	1	0	91	1	0	1
2	0	1	34	0	1	0
3	1	0	8	0	1	0
4	2	1	88	1	0	1

```
In [103... sns.heatmap(df_encoded.corr(), annot=True, fmt=".2f")
```

```
Out[103... <Axes: >
```



```
In [106... print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : <StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
contract_type : <StringArray>
['Annual', 'Monthly']
Length: 2, dtype: str
churn_risk : <StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```



```
In [107... # Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk']]

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories=order)
```

```
In [108... df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalations']]
```

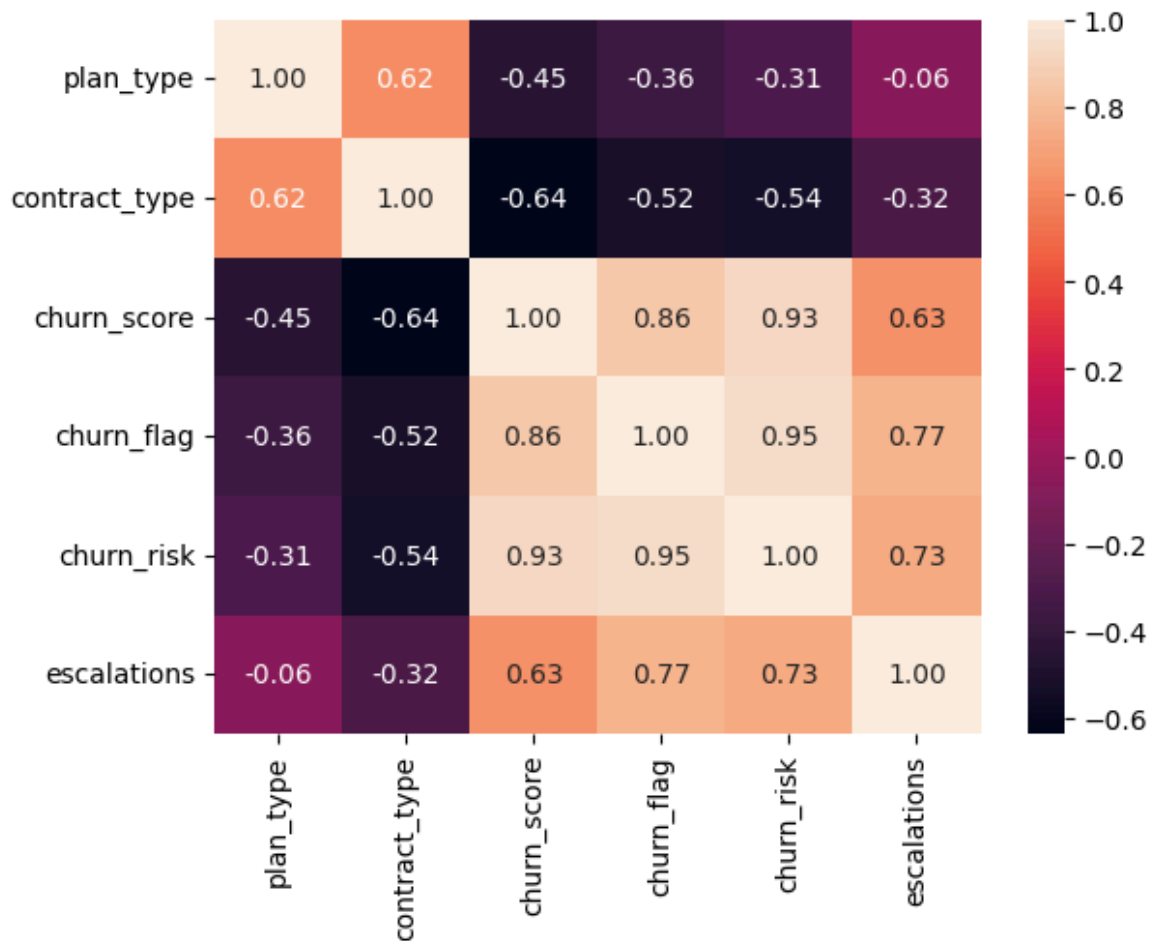
```
Out[108...
   plan_type  contract_type  churn_score  churn_flag  churn_risk  escalations
0  Standard      Annual         12          0         low          0
1  Premium      Annual         91          1         high          1
2   Basic      Monthly         34          0         low          0
3  Premium      Annual          8          0         low          0
4  Standard      Monthly        88          1         high          1
```

```
In [109... df_encoded.head()
```

```
Out[109...
   plan_type  contract_type  churn_score  churn_flag  churn_risk  escalations
0          1              1          12           0           0           0
1          2              1          91           1           2           1
2          0              0          34           0           0           0
3          2              1           8           0           0           0
4          1              0          88           1           2           1
```

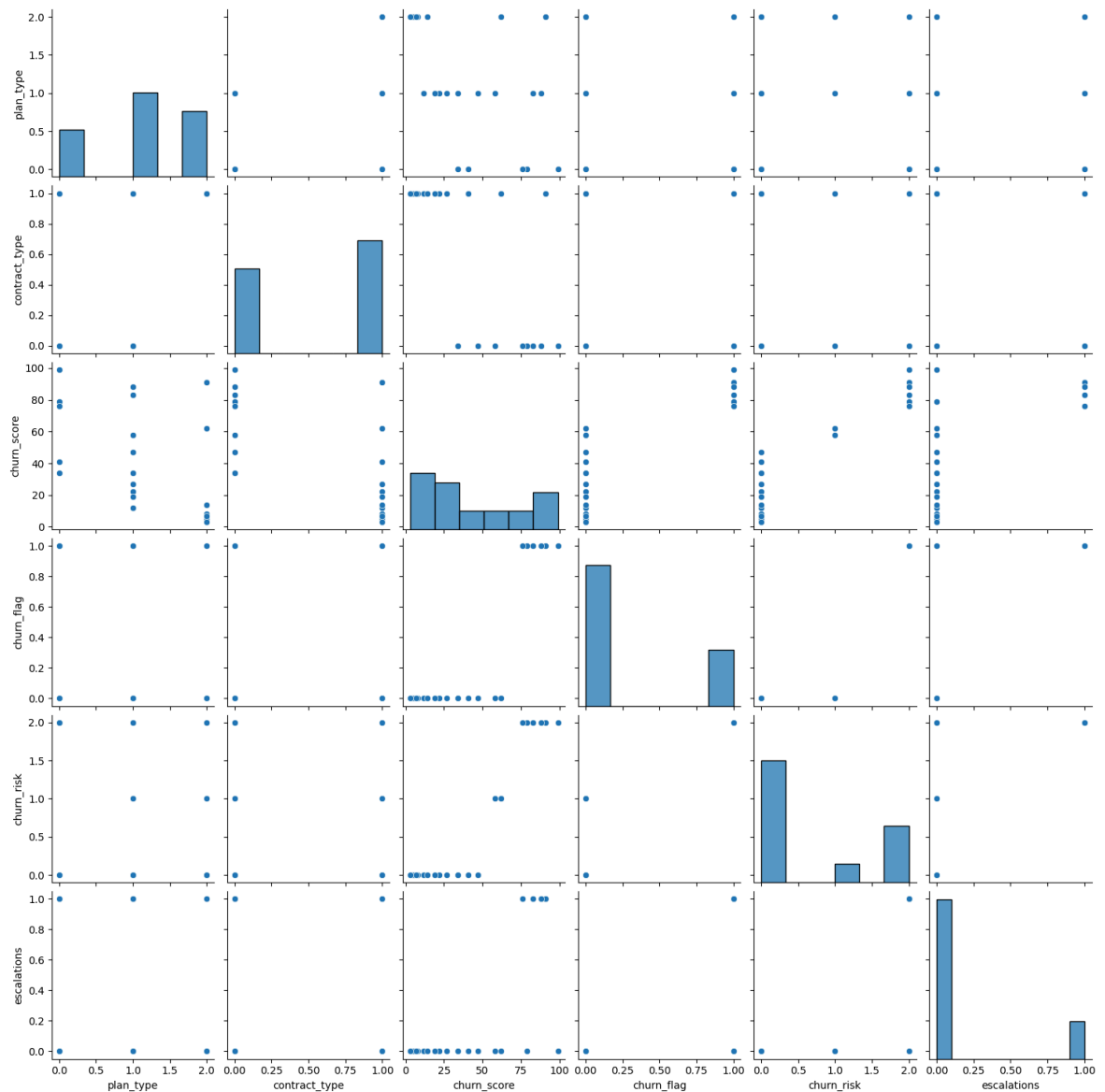
```
In [110... sns.heatmap(df_encoded.corr(), annot=True, fmt=".2f")
```

```
Out[110... <Axes: >
```



```
In [111... # pairplot - Plot pairwise relationships in a dataset
sns.pairplot(df_encoded)
```

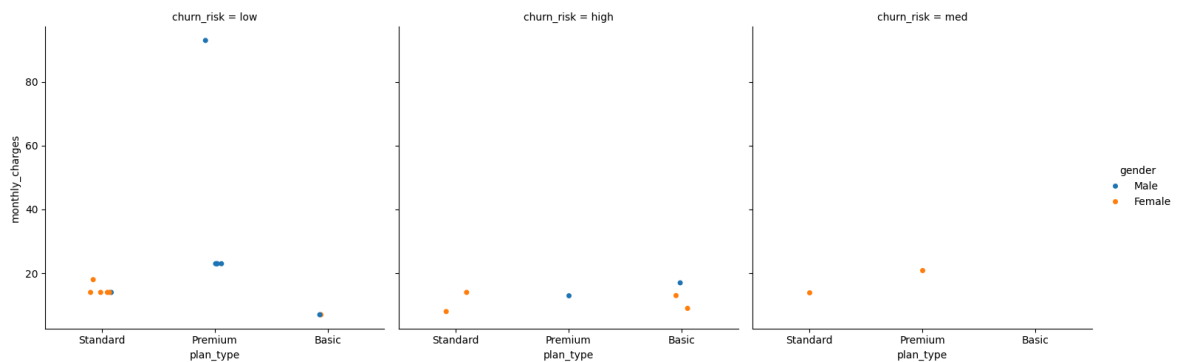
```
Out[111... <seaborn.axisgrid.PairGrid at 0x267fddf01a0>
```



In []: *# catplt/Facegrid plot - multi-dim comparison*

```
sns.catplot(data=df_visual,
             x='plan_type',
             y='monthly_charges',
             hue='gender',
             col='churn_risk')
```

Out[]: <seaborn.axisgrid.FacetGrid at 0x267ff9bc050>



Pivot table

```
In [113... pd.pivot_table(df_visual, index='plan_type', values='churn_flag', aggfunc = 'mea
```

```
Out[113... churn_flag
```

plan_type	
Basic	0.600000
Premium	0.142857
Standard	0.222222

```
In [ ]: # pivot table using multiple cols and agg type

pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
)
```

```
Out[ ]: churn_flag customerid monthly_charges
```

plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91